REFERENCE · CONCEPTS FIRST, PRODUCTS SECOND

The Platform-Agnostic Architecture

Every box of the governed agent-to-database architecture named by its role, with an open-source implementation and the Google Cloud guardrail side by side, the wire contract for every hop, and the same map for AWS and Azure.

The architecture in platform-agnostic terms

Agent Runtime, Agent Gateway, Model Armor and Private Service Connect are Google Cloud products. The reasons they exist are not Google-specific. Behind every one of them is a concept that any team can build from open-source parts: a reverse proxy in front of the agent, a forward proxy behind it, a content guardrail the proxies consult, a workload identity for the agent, a token exchange that narrows the user's authority, a private network link, a parameterized tool server, and a database that enforces rows and columns itself.

Figure 0A draws the architecture with those concepts only. Inside every box, a cyan line names an open-source implementation (Envoy Proxy is used for both proxies) and a green line names the Google Cloud guardrail that takes the same role in Figure 4 of Episode 01. The chapter badge points to where the box is introduced. Reading order is the same as the hop numbers: callers enter at hop 1, the agent exits at hop 4, and nothing reaches the database except hop 9.

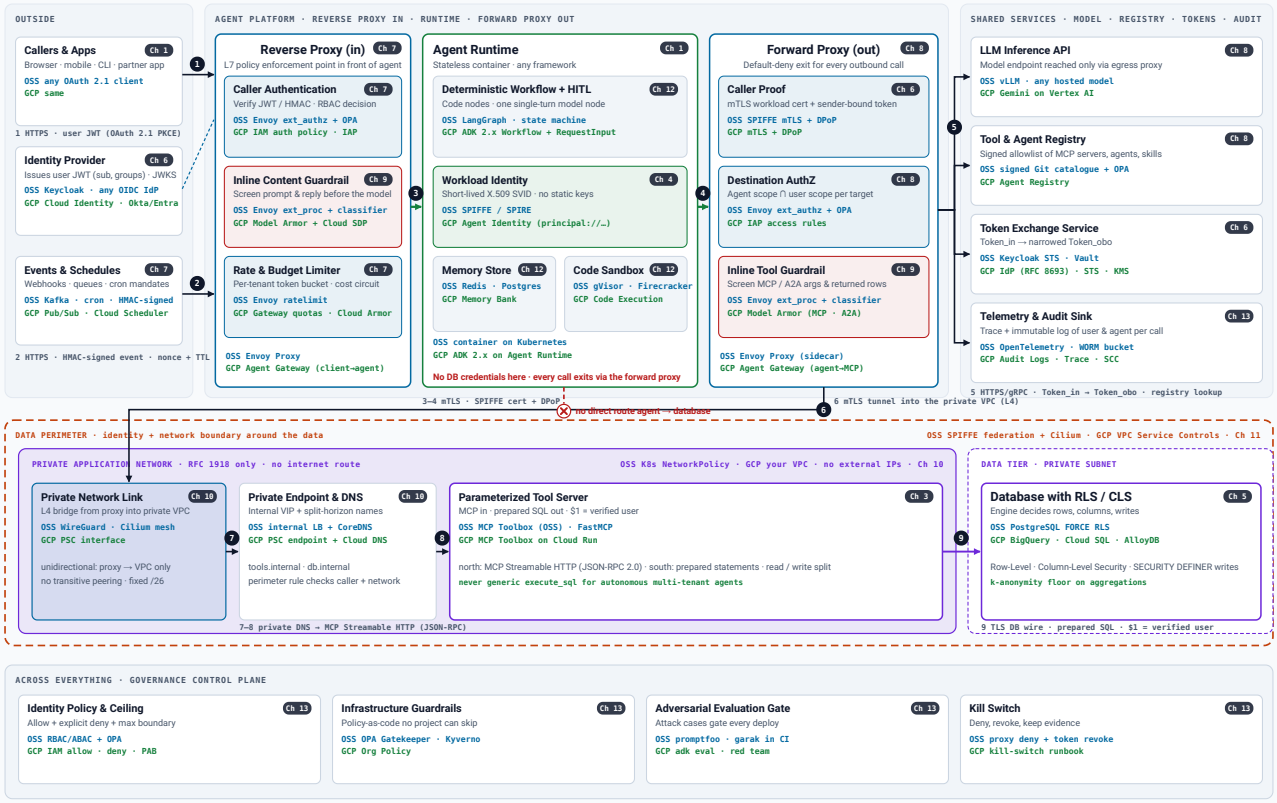


Figure 0A — The platform-agnostic decomposition. Every box is named by its role; the two lines inside each box name an open-source implementation (**OSS**, cyan) and the Google Cloud guardrail (**GCP**, green) that plays that role in Figure 4 of Episode 01. The chapter badge tells you where the box is explained. Hop numbers 1–9 refer to this figure.

Proxy enforcement points Inline content guardrail

Agent runtime & workload identity Private network, tool server, database

Data perimeter

How "Model Armor" looks without the product name

The red boxes in Figure 0A are the one component that is easiest to misread as a Google-only feature. It is not: the proxy pauses a request or response, streams the body to an external processor over gRPC, and only forwards when the processor returns a verdict. Figure 0A-2 shows the four stages that processor runs. On Google Cloud the processor is Model Armor with Cloud Sensitive Data Protection; in the open-source stack it is an Envoy `ext_proc` callout to a classifier such as Llama Guard with a de-identification library such as Presidio.

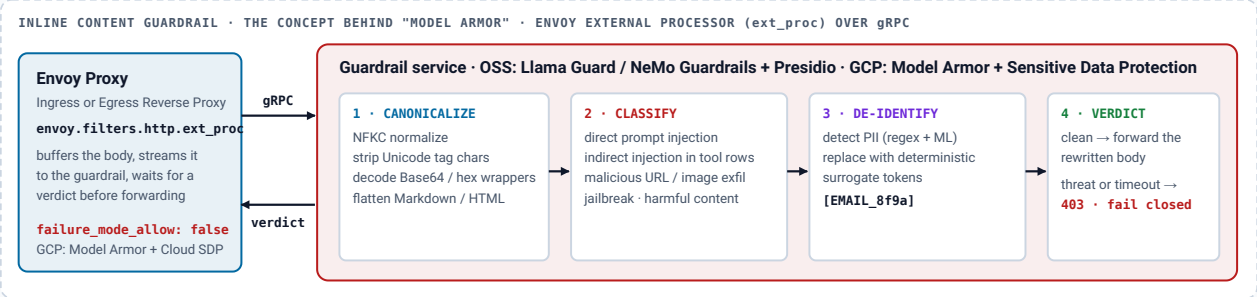


Figure 0A-2 — The Inline Content Guardrail is an out-of-process service the proxy consults on every request and response. On Google Cloud that service is Model Armor (with Cloud SDP for de-identification); in the open-source stack it is an Envoy ext_proc callout to a classifier.

Concept → open-source → Google Cloud: the master map

Every row below is one box in Figure 0A. The last two columns tell you which box it becomes in Figure 4 of Episode 01 and which chapter introduces it. The same strip repeats at the top of each chapter so the mapping is never more than one scroll away.

CONCEPT (FIGURE 0A BOX)	WHAT IT DOES	OPEN-SOURCE IMPLEMENTATION	GOOGLE CLOUD GUARDRAIL	BOX IN FIGURE 4 (EPISODE 01)	EXPLAINED IN
Callers & Apps	Browser · mobile · CLI · partner app	OSS Any OAuth 2.1 client (PKCE)	GCP Same (OAuth token)	`Customer Chat UI`	Ch. 1
Identity Provider	Issues user JWT (sub, groups) · JWKS	OSS Keycloak · Dex · any OIDC IdP	GCP Cloud Identity · Okta/Entra via Workforce IdF	`Corporate Sign-In`	Ch. 6
Events & Schedules	Webhooks · queues · cron mandates	OSS Kafka · NATS · cron (HMAC-signed)	GCP Pub/Sub · Cloud Scheduler · Eventarc	— (verified at the `Ingress Gate`; event-driven callers are covered in Episode 3)	Ch. 7
Reverse Proxy (in)	L7 policy enforcement point in front of agent	OSS Envoy Proxy	GCP Agent Gateway (CLIENT_TO_AGENT)	`Ingress Gate`	Ch. 7
Caller Authentication	Verify JWT / HMAC · RBAC decision	OSS Envoy ext_authz + OPA	GCP IAM auth policy · IAP · Cloud Armor	`Who may invoke`	Ch. 7

CONCEPT (FIGURE 0A BOX)	WHAT IT DOES	OPEN-SOURCE IMPLEMENTATION	GOOGLE CLOUD GUARDRAIL	BOX IN FIGURE 4 (EPISODE 01)	EXPLAINED IN
Inline Content Guardrail	Screen prompt & reply before the model	OSS Envoy ext_proc + Llama Guard + Presidio	GCP Model Armor + Cloud SDP	`Model Armor`	Ch. 9
Rate & Budget Limiter	Per-tenant token bucket · cost circuit	OSS Envoy ratelimit (Redis)	GCP Agent Gateway quotas · Cloud Armor	`Rate Quotas`	Ch. 7
Agent Runtime	Stateless container · any framework	OSS Container on Kubernetes	GCP ADK 2.x on Agent Runtime / Cloud Run	`2. AI Agent · ADK 2.x`	Ch. 1
Deterministic Workflow + HITL	Code nodes · one single-turn model node	OSS LangGraph · state machine	GCP ADK 2.x Workflow + RequestInput	`Workflow & Quarantine`	Ch. 12
Workload Identity	Short-lived X.509 SVID · no static keys	OSS SPIFFE / SPIRE	GCP Agent Identity (principal://…/AGENT_ID)	`Agent Identity · SPIFFE`	Ch. 4
Memory Store	Sessions keyed by (tenant, user)	OSS Redis · Postgres	GCP Memory Bank	`OOB Session`	Ch. 12
Code Sandbox	Generated code · no network · ephemeral	OSS gVisor · Firecracker	GCP Agent Runtime Code Execution	`Code Sandbox`	Ch. 12
Forward Proxy (out)	Default-deny exit for every outbound call	OSS Envoy Proxy (sidecar / egress gateway)	GCP Agent Gateway (AGENT_TO_MCP) · Envoy sidecar	`Egress Gate`	Ch. 8
Caller Proof	mTLS workload cert + sender-bound token	OSS SPIFFE mTLS + DPoP (RFC 9449)	GCP mTLS + DPoP	`Who is calling`	Ch. 6
Destination AuthZ	Agent scope ∩ user scope per target	OSS Envoy ext_authz + OPA	GCP IAP access rules (CEL)	`May it call this`	Ch. 8
Inline Tool Guardrail	Screen MCP / A2A args & returned rows	OSS Envoy ext_proc + Llama Guard	GCP Model Armor (MCP and A2A)	`Model Armor + SDP`	Ch. 9
LLM Inference API	Model endpoint reached only via egress proxy	OSS vLLM · Ollama · any hosted model	GCP Gemini on Vertex AI	`Gemini on Vertex AI`	Ch. 8
Tool & Agent Registry	Signed allowlist of MCP servers, agents, skills	OSS Git catalogue + OPA bundle · Backstage	GCP Agent Registry	`Agent Registry`	Ch. 8

CONCEPT (FIGURE 0A BOX)	WHAT IT DOES	OPEN-SOURCE IMPLEMENTATION	GOOGLE CLOUD GUARDRAIL	BOX IN FIGURE 4 (EPISODE 01)	EXPLAINED IN
Token Exchange Service	User token `Token_in` → narrowed tool token `Token_obo`	OSS Keycloak STS (RFC 8693) · Vault Transit	GCP IdP token endpoint (RFC 8693) · Google Cloud STS · Cloud KMS	`Token Exchange · IdP / STS`	Ch. 6
Telemetry & Audit Sink	Trace + immutable log of user & agent per call	OSS OpenTelemetry · Loki/Tempo · WORM bucket	GCP Cloud Audit Logs · Cloud Trace · SCC	`Audit Logs · BQ Analytics · SCC`	Ch. 13
Private Application Network	RFC 1918 only · no internet route	OSS Kubernetes NetworkPolicy · private subnets	GCP Your VPC (no external IPs / NAT)	`YOUR VPC`	Ch. 10
Private Network Link	L4 bridge from proxy into private VPC	OSS WireGuard · Cilium ClusterMesh	GCP PSC interface (network attachment)	`PSC Network Attachment`	Ch. 10
Private Endpoint & DNS	Internal VIP + split-horizon names	OSS Internal LB + CoreDNS	GCP PSC endpoint + Cloud DNS private zone	`PSC Endpoint & DNS`	Ch. 10
Parameterized Tool Server	MCP in · prepared SQL out · $1 = verified user	OSS MCP Toolbox for Databases (OSS) · FastMCP	GCP MCP Toolbox on Cloud Run (internal ingress)	`MCP Toolbox`	Ch. 3
Database with RLS / CLS	Engine decides rows, columns, writes	OSS PostgreSQL FORCE RLS · column GRANTs	GCP BigQuery · Cloud SQL · AlloyDB	`3. Database`	Ch. 5
Data Perimeter	Identity + network boundary around data	OSS SPIFFE federation + Cilium identity policy	GCP VPC Service Controls	`VPC SERVICE CONTROLS PERIMETER`	Ch. 11
Identity Policy & Ceiling	Allow + explicit deny + max boundary	OSS RBAC/ABAC + OPA constraints	GCP IAM allow, deny, PAB	`IAM Allow, Deny & PAB`	Ch. 13
Infrastructure Guardrails	Policy-as-code no project can skip	OSS OPA Gatekeeper · Kyverno	GCP Org Policy	`Org Policy Constraints`	Ch. 13
Adversarial Evaluation Gate	Attack cases gate every deploy	OSS promptfoo · garak · CI job	GCP adk eval · Evaluation and red team	`ADK Eval & Red Team CI`	Ch. 13
Kill Switch	Deny, revoke, keep evidence	OSS Proxy deny rule + token revoke	GCP Kill switch runbook	`3-Step Kill Switch`	Ch. 13

What travels on each wire (hops 1–9)

The arrows are contracts, not just lines. Each hop has a transport, an identity that must be on the wire, and a check the receiver must perform. Hop numbers refer to Figure 0A.

HOP	FROM → TO (FIGURE 0A NAMES AND NUMBERING)	TRANSPORT & WIRE PROTOCOL	IDENTITY ON THE WIRE	WHAT THE RECEIVER MUST VERIFY
Hop 1	**Callers & Client Apps → Ingress Reverse Proxy**	HTTPS (TLS 1.3) · HTTP/2 · carries `traceparent`	End-user OAuth 2.1 / OIDC JWT (`Token_in`) from Identity Provider via Authorization Code + PKCE	Caller Authentication (`ext_authz`) validates signature against IdP JWKS, `iss` , `aud` , `exp` ; Rate & Budget Limiter applies tenant quota.
Hop 2	**Event & Schedule Sources → Ingress Reverse Proxy**	HTTPS push · JSON event envelope	Detached HMAC-SHA256 (`X-Signature` , `X-Timestamp` , `X-Nonce`) or short-lived scheduler OIDC JWT	Recompute HMAC over raw body; reject if older than 300 s or nonce already seen; validate schema before the agent wakes up.
Hop 3	**Ingress Reverse Proxy → Agent Runtime**	Loopback / Unix socket (sidecar) or mTLS HTTP/2 (gateway)	Proxy workload cert + proxy-signed internal header with verified `user_sub` , `tenant_id`	Runtime accepts only the proxy's identity; strips external headers; receives the already screened and de-identified prompt.
Hop 4	**Agent Runtime → Egress Forward Proxy**	mTLS 1.3 over TCP (or UDS with peer check) · HTTP/2	Agent workload cert (SPIFFE SVID) + DPoP proof (RFC 9449) + user token `Token_in`	Caller Proof: extract SAN URI `spiffe://…` , verify DPoP signature matches key bound to token (`cnf.jkt`).
Hop 5	**Egress Forward Proxy → Registry · Token Exchange · LLM · Audit**	gRPC / HTTPS over TLS 1.3	Registry: `(agent_id, user_sub, target, tool)` tuple · STS: `subject_token=Token_in` , `actor_token=SVID` , `audience=tools` · LLM: platform model token injected by proxy	Destination Authorization = Agent scope ∩ User scope against the signed registry; STS returns 5-minute, audience-bound `Token_obo` with `act` claim; model output streams back through the guardrail.
Hop 6	**Egress Forward Proxy → Private Network Link**	TCP over L4 private tunnel · carries outer mTLS session	Source pinned to the link's /26 range + proxy workload cert	Only proxy-initiated connections cross; unsolicited connections back into the agent platform are dropped.
Hop 7	**Private Network Link → Private Endpoint & DNS**	Private L4 routing (RFC 1918) + split-horizon DNS	Network provenance (VPC + attachment id) + caller identity	Data Perimeter ingress rule requires both an allowed identity and an allowed source network.

HOP	FROM → TO (FIGURE 0A NAMES AND NUMBERING)	TRANSPORT & WIRE PROTOCOL	IDENTITY ON THE WIRE	WHAT THE RECEIVER MUST VERIFY
`Hop 8`	**Private Endpoint → Parameterized Tool Server**	MCP Streamable HTTP (HTTP/2 POST + JSON-RPC 2.0) over mTLS	Transport: proxy client cert · Header: downscoped `Token_obo` (or KMS-signed context JWT for jobs)	Tool server checks `aud`, extracts `verified_sub = Token_obo.sub`, validates arguments against JSON Schema; the model never supplies the user id.
`Hop 9`	**Parameterized Tool Server → Database**	Warehouse: HTTPS/gRPC per query · OLTP: TLS 1.3 on the DB wire protocol via a pool	Warehouse: per-query bearer `Token_obo` · OLTP: short-lived IAM DB token for the tool-server identity + `$1 = verified_sub` (or `SET LOCAL`)	Database executes prepared statements only; Row-Level and Column-Level Security filter for `verified_sub`; writes go through `SECURITY DEFINER` procedures.
`Hop *`	**Every component → Telemetry & Audit Sink**	OTLP / gRPC over TLS 1.3 · append-only log stream	Workload identity with write-only permission	One `trace_id` across all hops; every record carries both `agent_id` and `user_sub`.

The same map for other platforms

Because the concepts are stable, the substitution is mechanical. Google Cloud is the reference used in the episode; the other columns show that nothing in the design depends on it.

CONCEPT (FIGURE 0A)	OPEN-SOURCE / SELF-HOSTED	GOOGLE CLOUD (FIGURE 4, EPISODE 01)	AWS	AZURE
Ingress Reverse Proxy + Caller Authentication + Limiter	Envoy + OPA ext_authz + Redis ratelimit	**Agent Gateway (CLIENT_TO_AGENT) + IAP + Cloud Armor**	Envoy on EKS + WAF + ALB OIDC	Envoy on AKS + App Gateway WAF + Entra EasyAuth
Inline Content Guardrail (both proxies)	Envoy ext_proc → Llama Guard / NeMo Guardrails + Presidio	**Model Armor + Cloud SDP**	Envoy ext_proc → Bedrock Guardrails + Macie	Envoy ext_proc → Azure AI Content Safety + Purview
Agent Runtime + Workload Identity + Sandbox	Kubernetes + SPIRE + gVisor	**ADK 2.x on Agent Runtime + Agent Identity + Code Execution**	Bedrock AgentCore / EKS + IRSA + Firecracker	Container Apps / AKS + Workload Identity + Dynamic Sessions
Egress Forward Proxy + Caller Proof + Destination AuthZ	Envoy sidecar + SPIFFE mTLS + DPoP + OPA	**Agent Gateway (AGENT_TO_MCP) + IAP CEL, or Cloud Run Envoy sidecar**	Envoy sidecar + VPC Lattice + Verified Permissions	Envoy sidecar + APIM self-hosted gateway + NSG

CONCEPT (FIGURE 0A)	OPEN-SOURCE / SELF-HOSTED	GOOGLE CLOUD (FIGURE 4, EPISODE 01)	AWS	AZURE
Tool & Agent Registry	Signed Git catalogue + OPA bundles / Backstage	**Agent Registry**	Service Catalog + Parameter Store	Azure API Center
Token Exchange Service	Keycloak / Ory Hydra (RFC 8693) + Vault Transit	**IdP token endpoint (RFC 8693) + Google Cloud STS + Cloud KMS**	AWS STS / Cognito + KMS Sign	Entra OBO flow + Key Vault HSM
Private Network Link + Private Endpoint & DNS	WireGuard / Cilium ClusterMesh + CoreDNS	**PSC interface + PSC endpoint + Cloud DNS private zone**	PrivateLink interface endpoint + Route 53 private zone	Private Link service + Private Endpoint + Private DNS zone
Data Perimeter	SPIFFE federation + Cilium identity-aware policy	**VPC Service Controls**	AWS data perimeter (SCP + VPC endpoint policies)	Network Security Perimeter + Conditional Access
Parameterized Tool Server	MCP Toolbox for Databases (OSS) / FastMCP	**MCP Toolbox on Cloud Run (internal ingress)**	MCP Toolbox on Fargate + RDS Proxy IAM auth	MCP Toolbox on Container Apps + Managed Identity
Database with RLS / CLS	PostgreSQL 16 FORCE RLS + column GRANTs + SECURITY DEFINER	**BigQuery (policy tags, row access policies) · Cloud SQL / AlloyDB (parameterized secure views, PG RLS)**	Redshift RLS/CLS · Aurora PostgreSQL RLS	Fabric / Synapse RLS + DDM · Azure PostgreSQL RLS
Governance Control Plane	OPA Gatekeeper / Kyverno + promptfoo / garak CI	**IAM allow/deny/PAB + Org Policy + adk eval + kill switch**	SCPs + Config + Bedrock evaluations	Azure Policy + Defender + AI Foundry evaluations

Back to the episode

Return to Figure 4 of Episode 01, where the Google Cloud column is switched on and built one box at a time from Chapter 3 to Chapter 13.